R-assignment-script.R

paige

2026-10-18

```
# Author     : Ryan Peterson
# Last Modified   : 08/31/2020
# Description: BIOS 6621 R introduction

##### OUTLINE #####

# Part 1: Intro to the R language (Basics)
  # Basic Arithmetic
  # Sequences
  # Storing scalars and vectors
  # Common Functions
  # Indexing
  # The ? function
  # Reading in data, attaching

# Part 2: Types of objects in R

# Part 3: Simple tables and graphs in R
  # Tables and bar charts
  # Contingency tables and stratified bar charts
  # Continuous Data in R
    # Histograms
    # One-Way Scatter plots
    # Box plots
    # Two-Way Scatter plots
    # Measures of center and spread

# Part 4: Exercises

###

###### PART 1: Basics ######

### Basic Arithmetic, using R as a calculator ###
4 + 5 - 24/6
```

```
## [1] 5
```

```
(6-4)*3
```

```
## [1] 6
```

```r
5^2
```

```
## [1] 25
```

```r
5**2
```

```
## [1] 25
```

```r
### Sequences ###
1:8 # This notation provides all integers between 1 and 8
```

```
## [1] 1 2 3 4 5 6 7 8
```

```r
# this seq() function provides a sequence from 1 to 10 by 2
seq(1,10,2)
```

```
## [1] 1 3 5 7 9
```

```r
###  Repeating ###

# General:  rep( x, times= , each= )
# Repeat each element using argument 'each'
# Repeat whole result using argument 'times'

rep( 10, times=5 )            # c(10,10,10,10,10)
```

```
## [1] 10 10 10 10 10
```

```r
rep( c(1,3,5), times=2 )      # c(1,3,5,1,3,5)
```

```
## [1] 1 3 5 1 3 5
```

```r
rep( c(1,3,5), each=2 )       # c(1,1,3,3,5,5)
```

```
## [1] 1 1 3 3 5 5
```

```r
### Storing scalars and vectors ###

x <- 1:9  # stores a vector
x         # prints out that vector
```

```
## [1] 1 2 3 4 5 6 7 8 9
```

```r
x <- 6    # stores a scalar
x         # prints out that scalar
```

```
## [1] 6
```

```r
x + 10    # arithmetics can be performed on stored values
```

```
## [1] 16
```

```r
x <- x + 10
x
```

```
## [1] 16
```

```r
### Common functions ###
x <- 1:9

mean(x) # Calculates the mean of what you stored in x
```

```
## [1] 5
```

```r
mean(1:9)
```

```
## [1] 5
```

```r
sd(x) # Calculates the standard deviation
```

```
## [1] 2.738613
```

```r
sum(x)
```

```
## [1] 45
```

```r
min(x)
```

```
## [1] 1
```

```r
max(x)
```

```
## [1] 9
```

```r
#this is the max of the vector x
max_x <- max(x)
max_x
```

```
## [1] 9
```

```r
avg <- mean(x)

# c() is a function that tells R to store a vector
x <- c(2,3,5)
x
```

```
## [1] 2 3 5
```

```r
### Indexing ###
# Say we want to extract the 2nd value in x. We use x[2]
b <- x[2]
b
```

```
## [1] 3
```

```r
### The ? funtion ###
# If you are unsure of a function or how to use it, use ?
# Help documentation will appear on the right
?max
?seq
seq(to = 10,from = 1, by = 2)
```

```
## [1] 1 3 5 7 9
```

```r
seq(10,1,-2) # this will yield an error
```

```
## [1] 10  8  6  4  2
```

```r
# More examples
seq( from=4, to=8, by=1 )      # c(4,5,6,7,8)   or ...
```

```
## [1] 4 5 6 7 8
```

```r
seq( 4, 8, by=1 )              # same
```

```
## [1] 4 5 6 7 8
```

```r
4:8                            # same, shorthand
```

```
## [1] 4 5 6 7 8
```

```r
seq( 0, 1, by=0.1 )            # c(0, 0.1, 0.2, ..., 0.9, 1.0)
```

```
##  [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```r
### Reading in data ###
# The following code will open up a file browser
# Download, then read in the "week01-assignment-data-titanic.csv" data set on Canvas
mydata <- read.csv("C:/Users/paige/OneDrive/Desktop/CSPH F26/BIOS 6621/Wagner_BIOS6621/DataRaw/R-assignm


# The head() function shows the first 6 observations
# of each variable, along with variable names
head(mydata)
```

```
##   Class  Sex   Age Survived
## 1   3rd Male Child     Died
## 2   3rd Male Child     Died
## 3   3rd Male Child     Died
## 4   3rd Male Child     Died
## 5   3rd Male Child     Died
## 6   3rd Male Child     Died
```

```
# Each observation is one person who was on the Titanic
# how many people were on the Titanic?

dim(mydata) # this gives you the dimensions of the data set
```

```
## [1] 2201    4
```

```
# Now I know there were 2201 people on the Titanic (how?)

# To work with the variables in mydata, use the $ operator
head(mydata$Class)
```

```
## [1] "3rd" "3rd" "3rd" "3rd" "3rd" "3rd"
```

```
##### PART 2: Types of objects, data management #####

### There are many kinds of objects in R.  Some of the most common are
#   vector
#   matrix
#   data frame (matrix where columns may be of different types (eg numeric or character)
#   list (set of common objects bound together with one name)

### Vectors

x <- c(1,3,4,6,9,10,11,14)                # numeric vectors
y1 <- c(102,101,89,92,81,80,75,72)
y2 <- c(88,85,81,80,76,71,66,64)
lab <- c("a","b","c","d","e","f","g","h")  # character vector

# Sub-vectors
x[c(1,3,7)]         # selects elements 1, 3, and 7 of x
```

```
## [1]  1  4 11
```

```
x[c(3:5)]           # selects elements 3,4,5 of x
```

```
## [1] 4 6 9
```

```
x[3:5]              # same
```

```
## [1] 4 6 9
```

```r
x[-4]               # x omitting element 4
```

```
## [1]  1  3  4  9 10 11 14
```

```r
x[-c(2,4,6)]        # x omitting elements 2,4,6
```

```
## [1]  1  4  9 11 14
```

```r
# Note that there is no <- in these, so the object is created and printed but not stored

# Vectors can be added, multiplied, logged, etc -- check the result on a small example
2*x
```

```
## [1]  2  6  8 12 18 20 22 28
```

```r
log(x)
```

```
## [1] 0.000000 1.098612 1.386294 1.791759 2.197225 2.302585 2.397895 2.639057
```

```r
x+y1
```

```
## [1] 103 104  93  98  90  90  86  86
```

```r
x*y1
```

```
## [1]  102  303  356  552  729  800  825 1008
```

```r
# Concatenating vectors
c(x,y1,y2)
```

```
##  [1]   1   3   4   6   9  10  11  14 102 101  89  92  81  80  75  72  88  85  81  80  76  71  66  64
```

```r
c(x, lab)           # note vectors are different types
```

```
##  [1] "1"  "3"  "4"  "6"  "9"  "10" "11" "14" "a"  "b"  "c"  "d"  "e"  "f"  "g"  "h"
```

```r
### Matrices

### First dimension is row (horizontal), 2nd dimension is column (vertical)

xy.mat <- cbind(x, y1, y2)              # cbind puts columns together
xy.long <- rbind(x, y1, y2)             # rbind stacks rows together
na.mat <- matrix( NA, ncol=4, nrow=2 )  # useful to create a blank matrix to fill in later

# Warnings will generally be given when dimensions don't match
cbind(x, c(1:5))
```

```
## Warning in cbind(x, c(1:5)): number of rows of result is not a multiple of vector length (arg 2)
```

```
##       x
## [1,]  1 1
## [2,]  3 2
## [3,]  4 3
## [4,]  6 4
## [5,]  9 5
## [6,] 10 1
## [7,] 11 2
## [8,] 14 3
```

```
# But maybe not ...  check, check, and check again (same in SAS)
cbind(x, c(1:4))
```

```
##       x
## [1,]  1 1
## [2,]  3 2
## [3,]  4 3
## [4,]  6 4
## [5,]  9 1
## [6,] 10 2
## [7,] 11 3
## [8,] 14 4
```

```
# Sub-matrices are selected the same way as sub-vectors,
#   using the methods above on each dimension
#   leaving a dimension blank means use all elements
xy.mat[2:4, ]          # rows 2, 3, 4, all columns
```

```
##      x  y1 y2
## [1,] 3 101 85
## [2,] 4  89 81
## [3,] 6  92 80
```

```
xy.mat[, -2]           # omit column 2
```

```
##       x y2
## [1,]  1 88
## [2,]  3 85
## [3,]  4 81
## [4,]  6 80
## [5,]  9 76
## [6,] 10 71
## [7,] 11 66
## [8,] 14 64
```

```
xy.mat[2:4, -2]
```

```
##      x y2
## [1,] 3 85
## [2,] 4 81
## [3,] 6 80
```

```r
xy.mat[c(1,3,7),]
```

```
##       x  y1 y2
## [1,]  1 102 88
## [2,]  4  89 81
## [3,] 11  75 66
```

```r
# Subsets can be formed from conditions
x[x > 10]
```

```
## [1] 11 14
```

```r
y1[x > 10]
```

```
## [1] 75 72
```

```r
xy.mat[x>10, ]
```

```
##       x y1 y2
## [1,] 11 75 66
## [2,] 14 72 64
```

```r
xy.mat[ x <= 10 & x > 4, ]
```

```
##       x y1 y2
## [1,]  6 92 80
## [2,]  9 81 76
## [3,] 10 80 71
```

```r
# Dimensions can be gotten using
length(x)
```

```
## [1] 8
```

```r
dim( xy.mat )
```

```
## [1] 8 3
```

```r
##### PART 3: Tables + Plots #####

### Tables and bar plots ###

# the table() function is straightforward
table(mydata$Class)
```

```
##
##  1st  2nd  3rd Crew
##  325  285  706  885
```

```r
table(mydata$Sex)
```

```
##
## Female   Male
##    470   1731
```

```r
table(mydata$Age)
```

```
##
## Adult Child
##  2092   109
```

```r
table(mydata$Survived)
```

```
##
##     Died Survived
##     1490      711
```

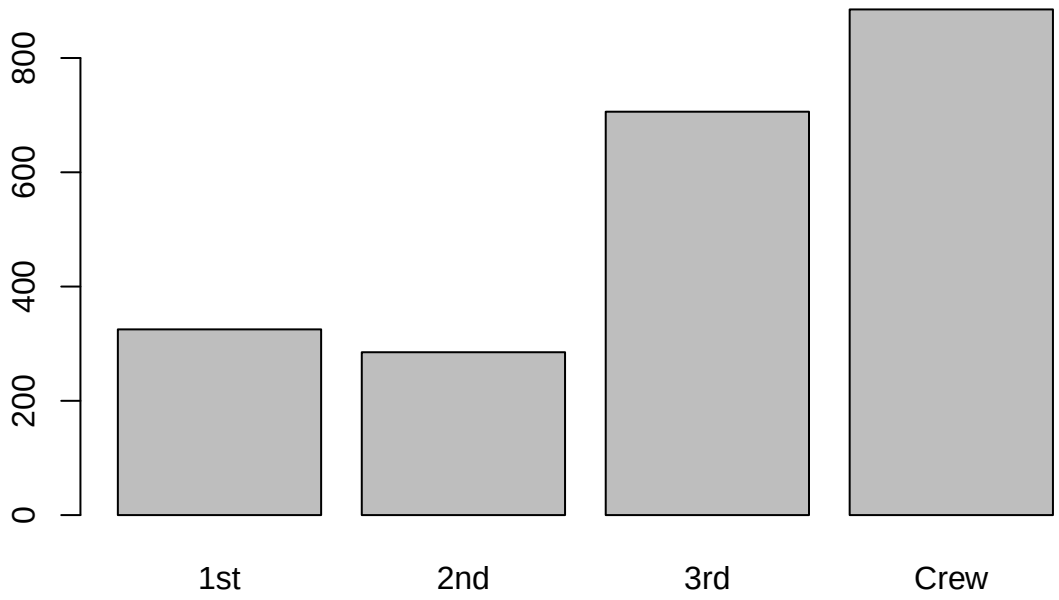
```r
# R allows you to store these tables
tab1 <- table(mydata$Class)
tab1
```

```
##
##  1st  2nd  3rd Crew
##  325  285  706  885
```

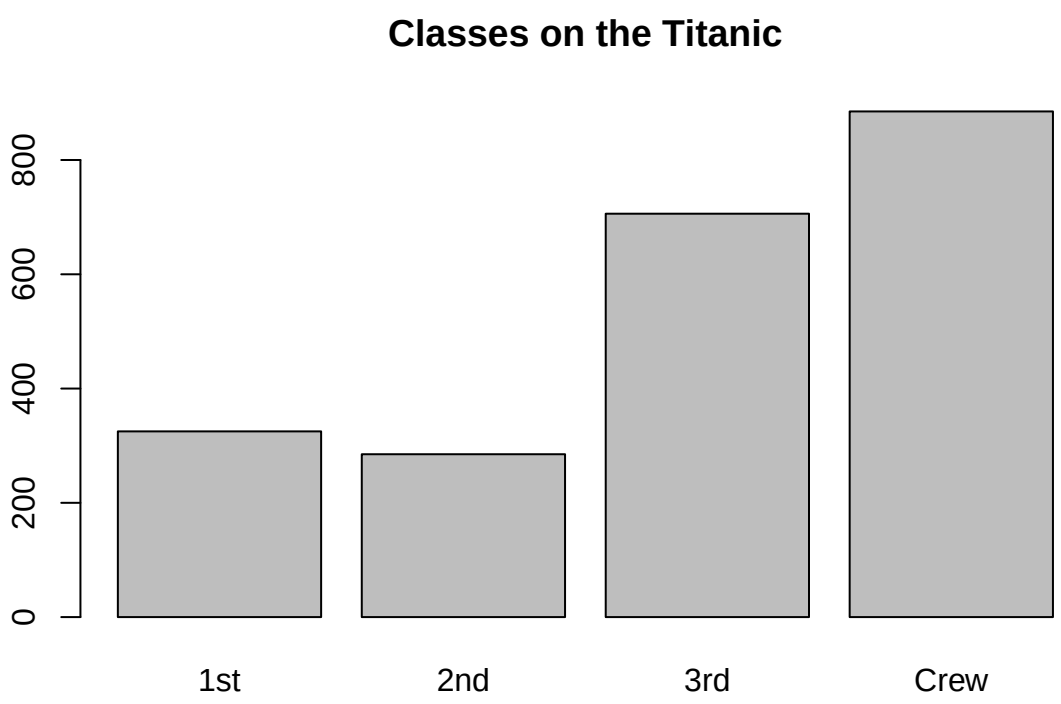
```r
# once you have a table...
# you can get the relative frequencies with prop.table()
prop.table(tab1)
```

```
##
##       1st       2nd       3rd      Crew
## 0.1476602 0.1294866 0.3207633 0.4020900
```

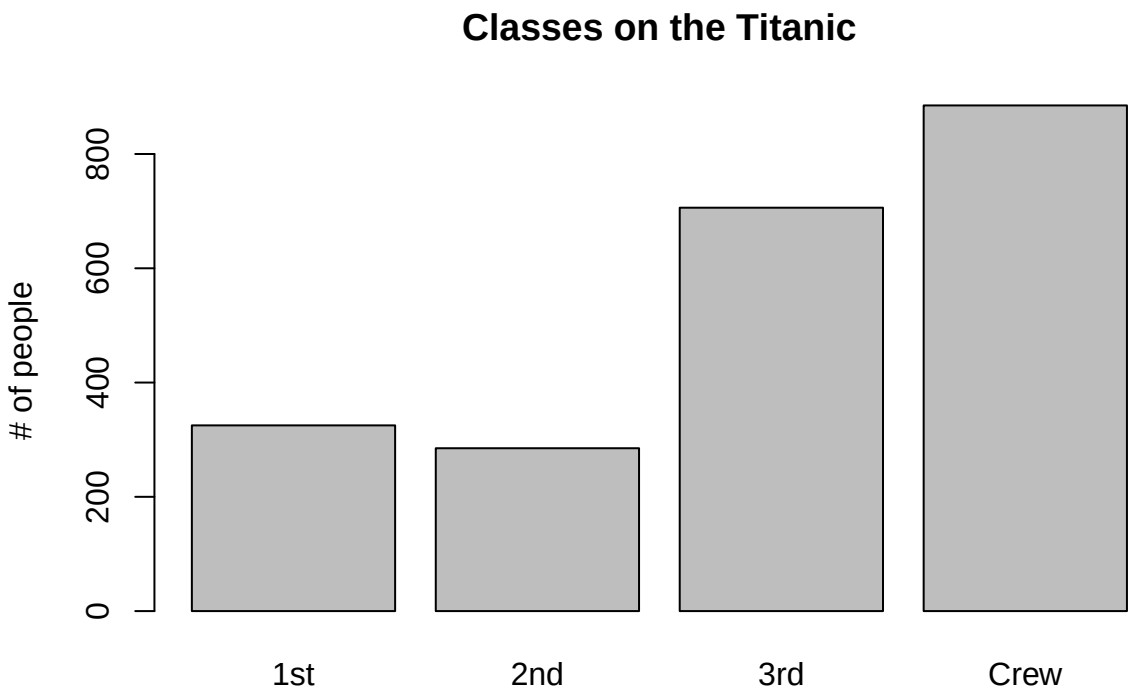
```r
# and you can create a bar chart with barplot()
barplot(tab1)
```



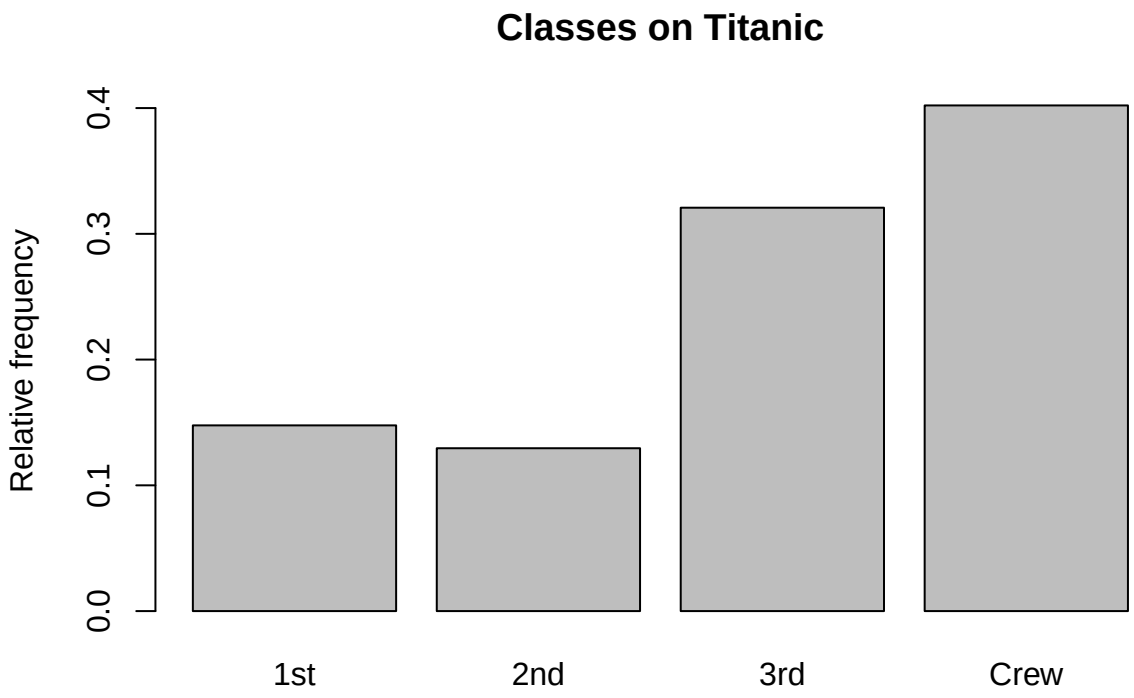

```
# how can we make this plot better?
?barplot
# notice that the "main" argument refers to an overall title for the plot
barplot(tab1, main = "Classes on the Titanic")
```




```
# let's add on a y-axis label
barplot(tab1, main = "Classes on the Titanic", ylab = "# of people")
```




```
# We can do relative frequencies as well
barplot(prop.table(tab1), main = "Classes on Titanic",
        ylab = "Relative frequency")
```




```
### Contingency tables and stratified bar charts ###

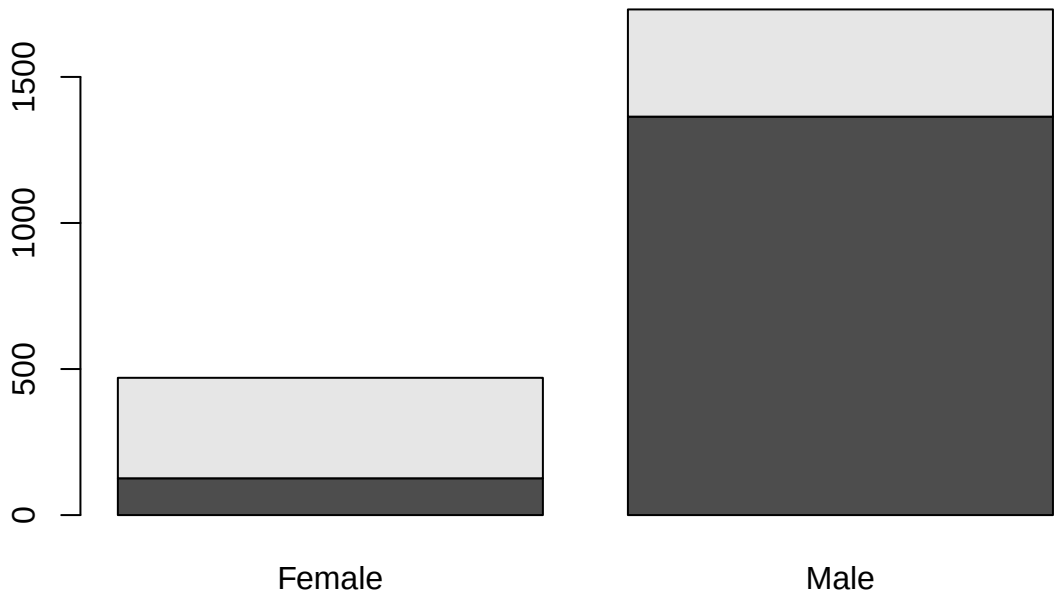
# the table() function also works for multiple variables
tab2 <- table(mydata$Sex,mydata$Survived)
tab2 # This is a contingency table
```

```
##
##          Died Survived
##   Female  126      344
##   Male   1364      367
```

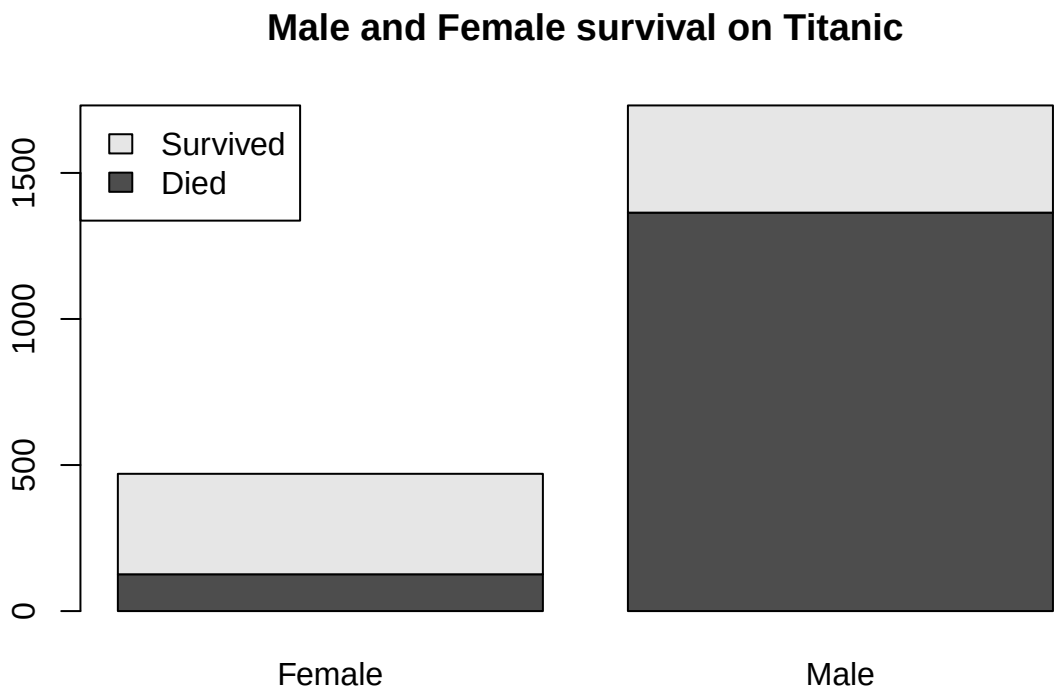
```
# barplot() works on these as well, but not perfectly
barplot(tab2)
```




```
# We'd prefer gender on the x axis, so let's flip our table
tab3 <- table(mydata$Survived, mydata$Sex)
barplot(tab3)
```



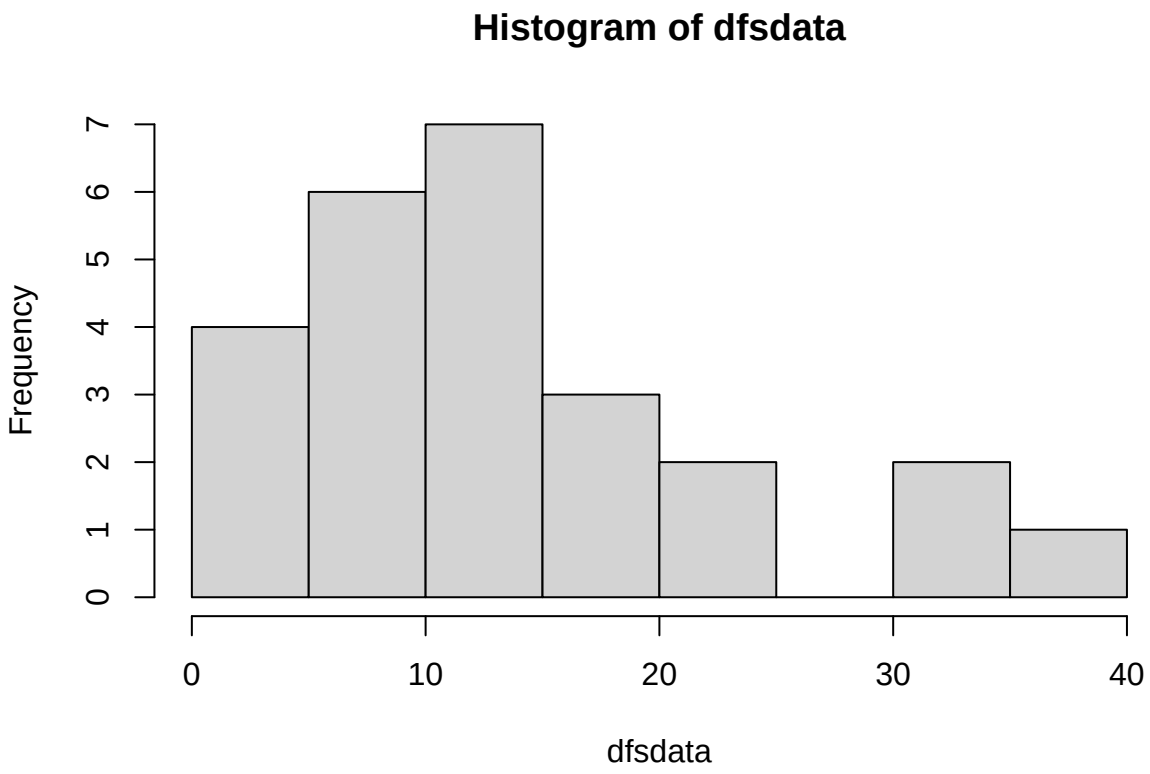

```
# we still need to add a title
# and, more importantly, a legend
barplot(tab3, main = "Male and Female survival on Titanic",
        legend.text = TRUE, args.legend = list(x = "topleft"))
```



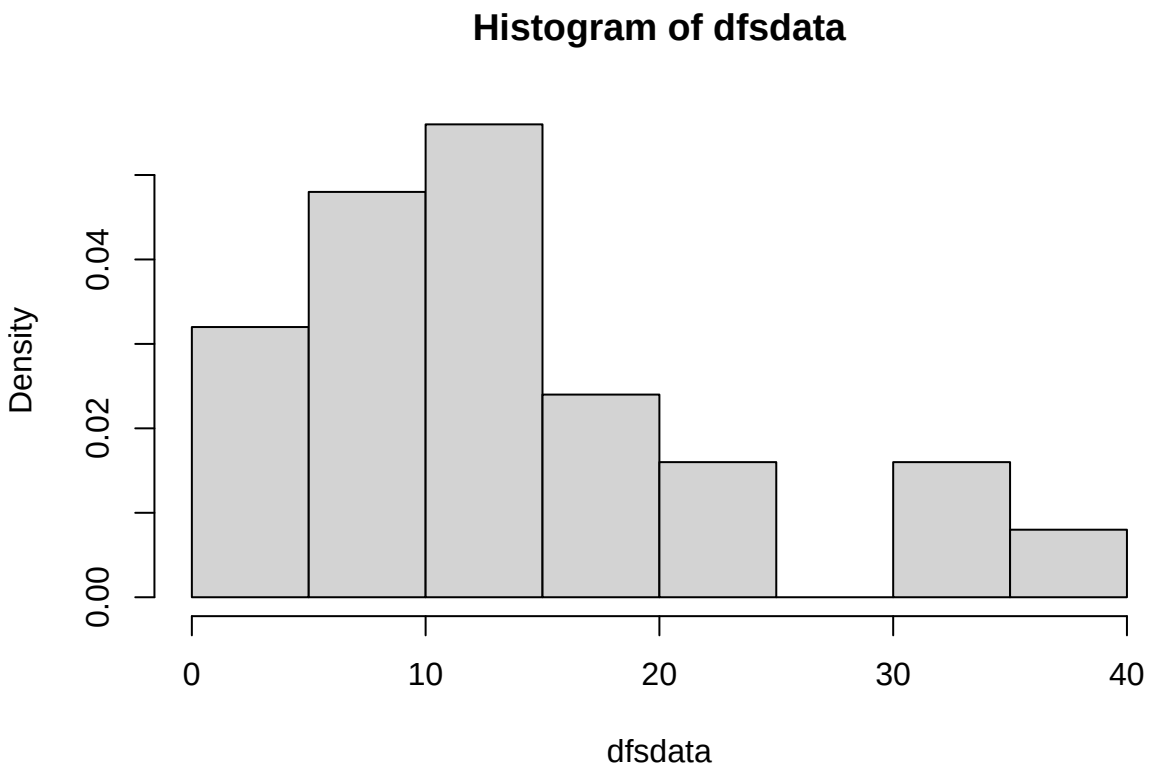

```
### Continuous Data in R ###

# The disease-free survival example
# Note: for the "disease-free survival" example:
#   This is a sample of 25 cancer patients
#   The DFS time of a treated cancer patient is defined as the length of
#   elapsed time between the time at which the patient goes into remission
#   and the time at which the patient relapses.

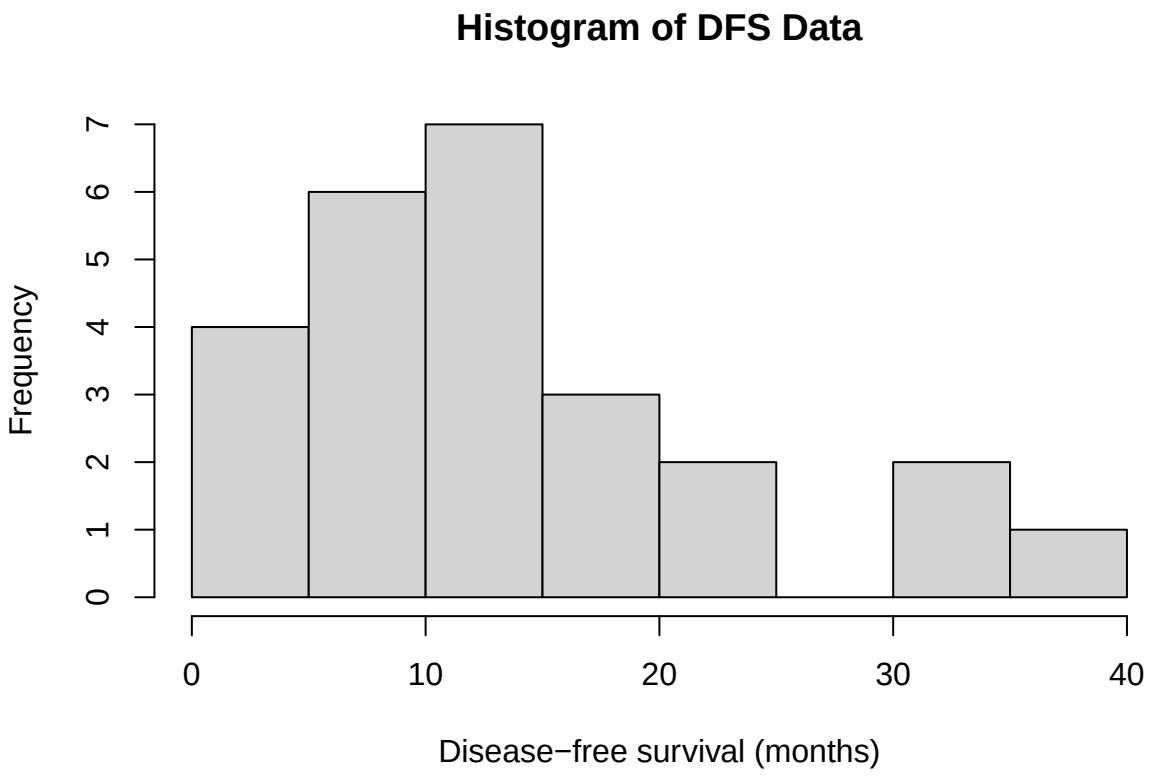
dfsdata <- c(1, 2, 3, 5, 7, 8, 8, 9, 10,
             10, 11, 11, 12, 12, 13, 14,
             15, 17, 18, 19, 21, 22, 34,
             35, 39)
hist(dfsdata) # absolute frequency histogram is default
```



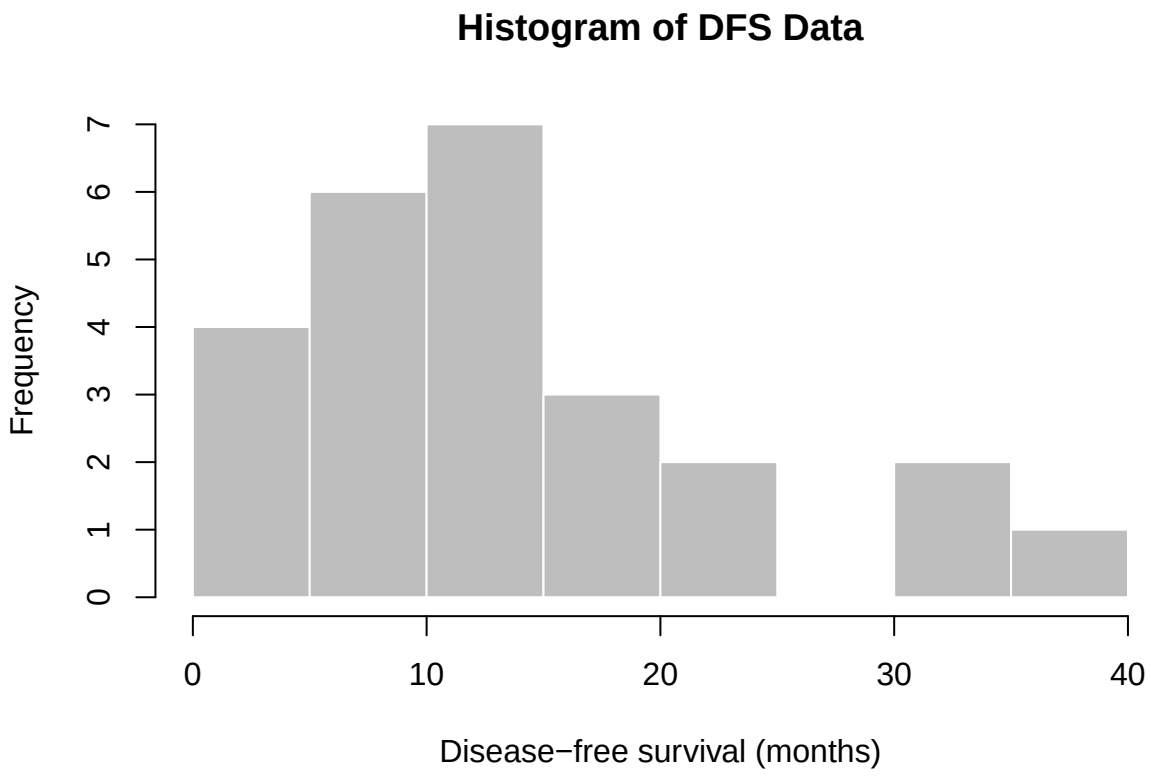

```r
hist(dfsdata, freq = F) # density histogram
```




```
# let's make it look a bit nicer
hist(dfsdata, main = "Histogram of DFS Data",
     xlab = "Disease-free survival (months)")
```

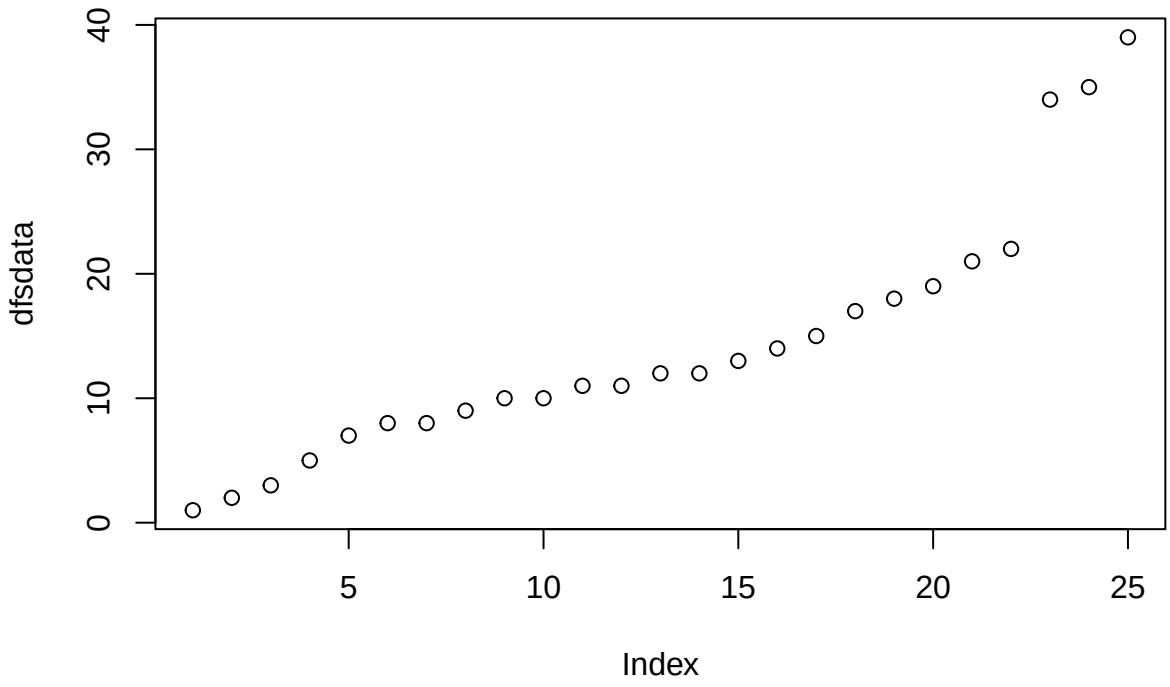



```
# Feel free to play around with other graphical parameters
hist(dfsdata, main = "Histogram of DFS Data",
     xlab = "Disease-free survival (months)",
     col = "grey", border = "white")
```



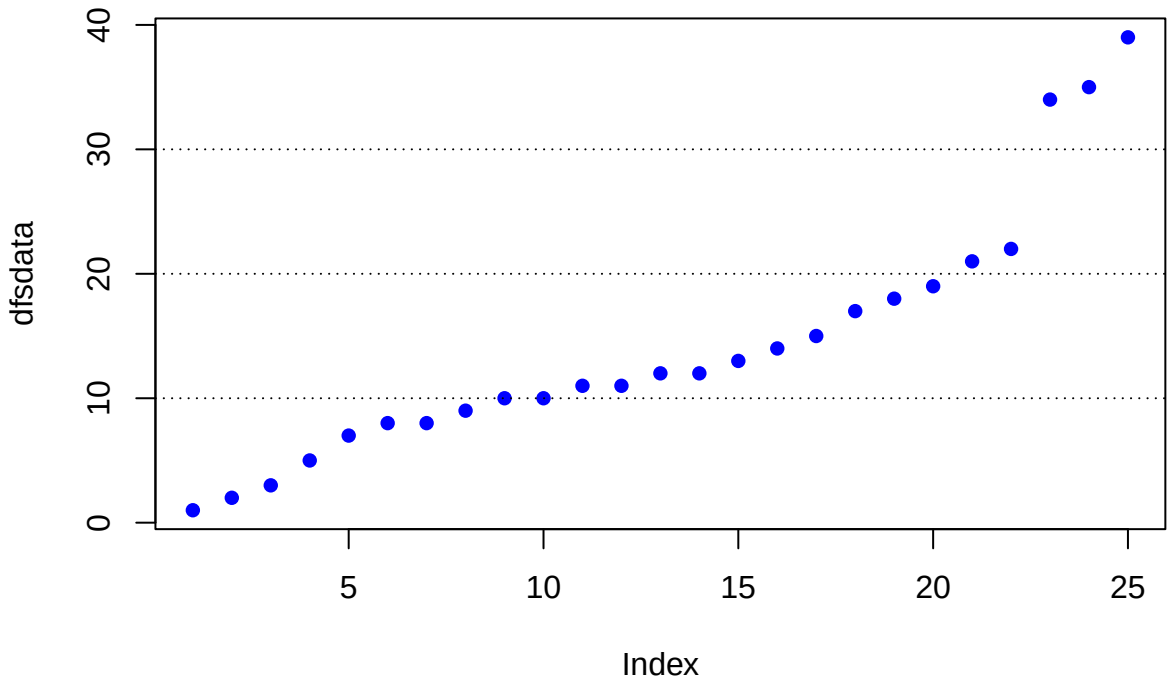

```
## One-Way Scatter plot ##

# You can create a scatter plot with plot()
plot(dfsdata) # the default x-axis is just an index
```




```
plot(dfsdata, pch = 16, col = "blue")
# For more pch options, check out the link below
# http://www.endmemo.com/program/R/pchsymbols.php

# We can add to plots using other functions
# to add gridlines, you can use abline()
abline(h = c(10, 20, 30), lty = 3)
```

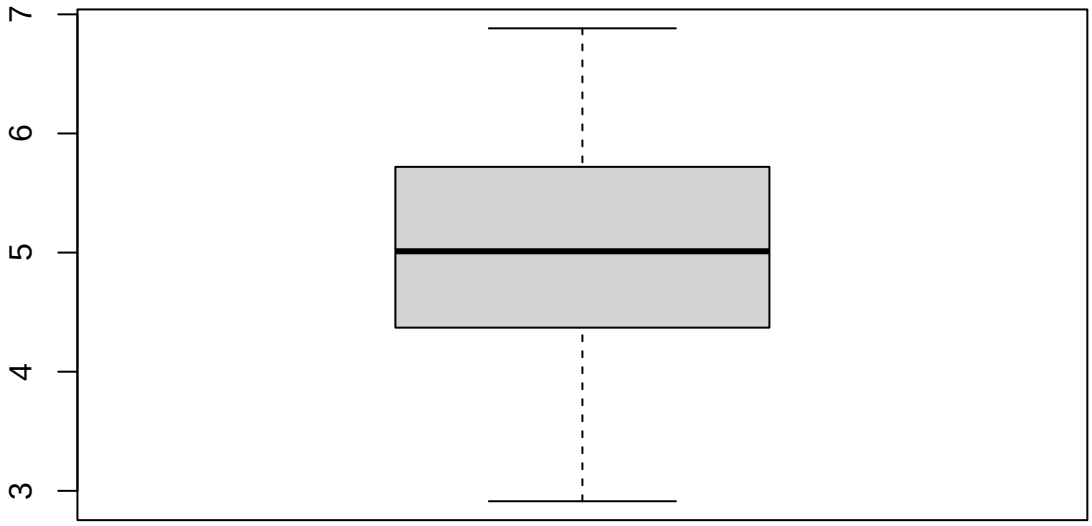



```
## Box Plot ##

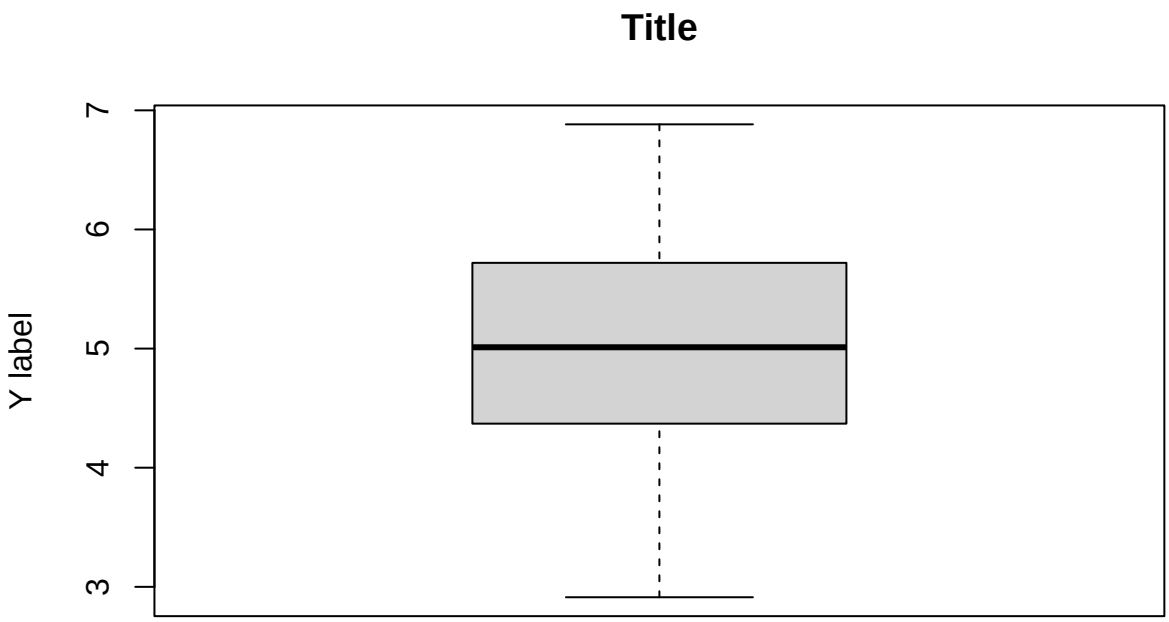
example <- rnorm(100,5) # This generates random numbers around 5
example
```

```
##   [1] 5.287167 5.082311 5.995863 4.186041 6.253488 5.748698 6.264486 3.249283 4.826240 5.868349 5.304
##  [12] 5.734146 5.063319 3.718684 5.337419 5.009086 4.447362 4.329969 5.445322 4.462718 5.217275 6.375
##  [23] 5.554130 5.012981 3.682008 5.799016 5.409971 5.748451 4.455781 4.935774 5.888286 2.912678 3.069
##  [34] 5.745184 4.376851 4.762466 4.709108 5.792459 5.494989 6.420076 5.697735 4.362870 4.008911 5.018
##  [45] 6.277023 3.753237 4.645475 4.246361 6.219434 5.054695 6.117319 6.882341 3.398797 5.824728 4.911
##  [56] 4.818025 4.505973 4.767695 4.814935 4.342955 5.290526 3.928557 4.136317 4.186602 4.733879 3.763
##  [67] 4.398485 3.334514 5.790007 5.174698 4.916327 4.639709 5.781391 4.399858 5.232605 4.217228 5.223
##  [78] 3.617156 5.708779 4.299429 5.245152 4.730934 6.145655 4.664523 4.118415 5.448317 6.573307 5.045
##  [89] 5.752515 3.680688 3.725082 6.546709 4.486846 5.213114 4.683195 5.590109 4.906413 4.112071 5.730
## [100] 5.109301
```

```
boxplot(example)
```




```
boxplot(example, main = "Title", ylab = "Y label")
```

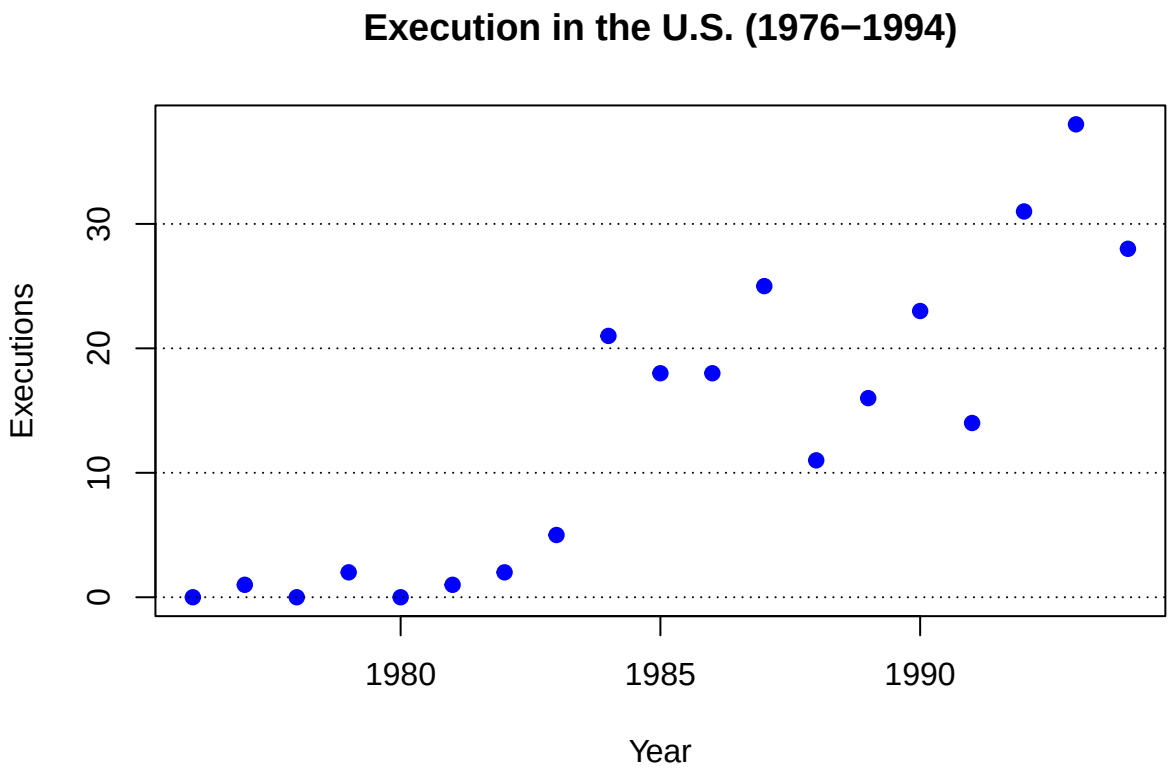



```
quantile(example) # returns important percentiles, 5 number summary
```

```
##       0%      25%      50%      75%     100%
## 2.912678 4.373355 5.011033 5.714290 6.882341
```

```
## Two-Way Scatter Plot ##

# This is the US execution data from class
Executions <- c(0,1,0,2,0,1,2,5,21,18,18,25,11,16,23,14,31,38,28)
Year <- 1976:1994
plot(Year, Executions, pch=19, main="Execution in the U.S. (1976-1994)", col="blue")
abline(h=10*0:3, lty=3)
```




```
# Some measures of center
mean(Executions)
```

```
## [1] 13.36842
```

```
median(Executions)
```

```
## [1] 14
```

```
table(Executions) # note the mode is 0
```

```
## Executions
##  0  1  2  5 11 14 16 18 21 23 25 28 31 38
##  3  2  2  1  1  1  1  2  1  1  1  1  1  1
```

```
# measures of spread
var(Executions) # Variance
```

```
## [1] 146.9123
```

```
sd(Executions)  # Standard Deviation
```

```
## [1] 12.12074
```

```r
quantile(Executions)
```

```
##   0%  25%  50%  75% 100%
##  0.0  1.5 14.0 22.0 38.0
```

```r
IQR <- 22.0 - 1.5
IQR             # Interquartile range
```

```
## [1] 20.5
```

```r
max(Executions) - min(Executions) # range
```

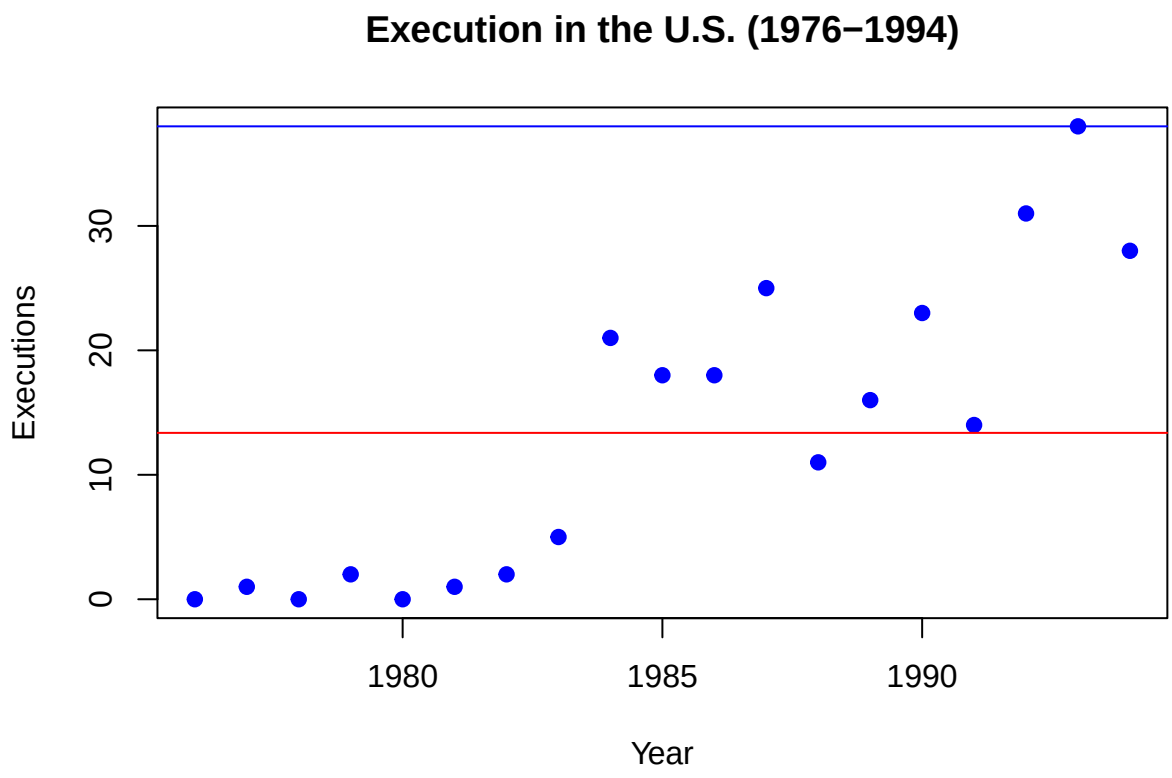
```
## [1] 38
```

```r
# how many people were executed in the US from 1976 - 1994?
sum(Executions)
```

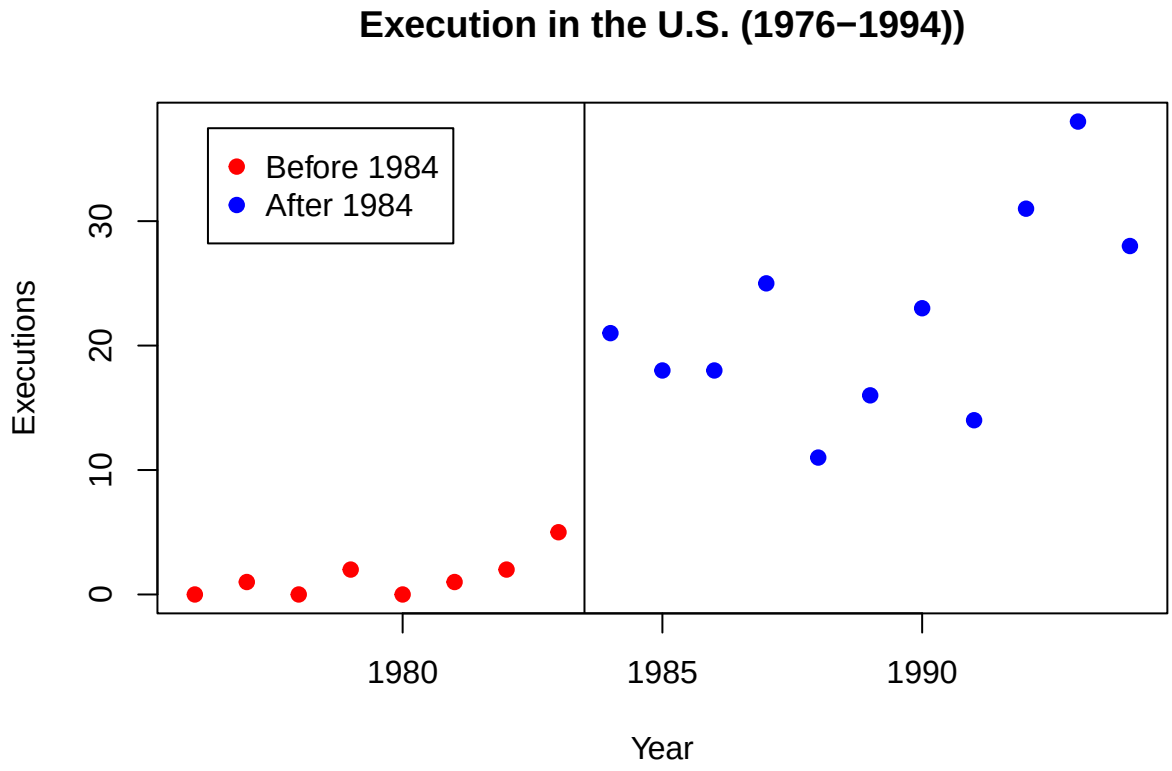
```
## [1] 254
```

```r
##### PART 4: Exercises #####

### Ex 1:  Add a red horizontal line at mean(Executions) and a blue
### horizontal line at max(Executions) to the plot below
plot(Year, Executions, pch=19, main="Execution in the U.S. (1976-1994)", col="blue")
abline(h=mean(Executions), col="red")
abline(h=max(Executions), col="blue")
```



```
### Ex 2:  Color the blue points red if Year<1984, add a legend to the plot,
###        and add a vertical line at Year 1983.5
plot(Year, Executions, pch=19, main="Execution in the U.S. (1976-1994))", col= ifelse(Year<1984, "red",
legend("topleft", inset= 0.05, legend = c("Before 1984", "After 1984"), pch=19, col=c("red","blue"))
abline(v=1983.5)
```




```
### Ex 3:  Use rep and seq to create c(.2,.2,.4,.4,.6,.6,.2,.2,.4,.4,.6,.6,.2,.2,.4,.4,.6.,6)
x <- c(.2,.4,.6)
rep(x, times=3, each=2)
```

```
##  [1] 0.2 0.2 0.4 0.4 0.6 0.6 0.2 0.2 0.4 0.4 0.6 0.6 0.2 0.2 0.4 0.4 0.6 0.6
```

```
seq(from=0.2, to=0.6, by=0.2)
```

```
## [1] 0.2 0.4 0.6
```

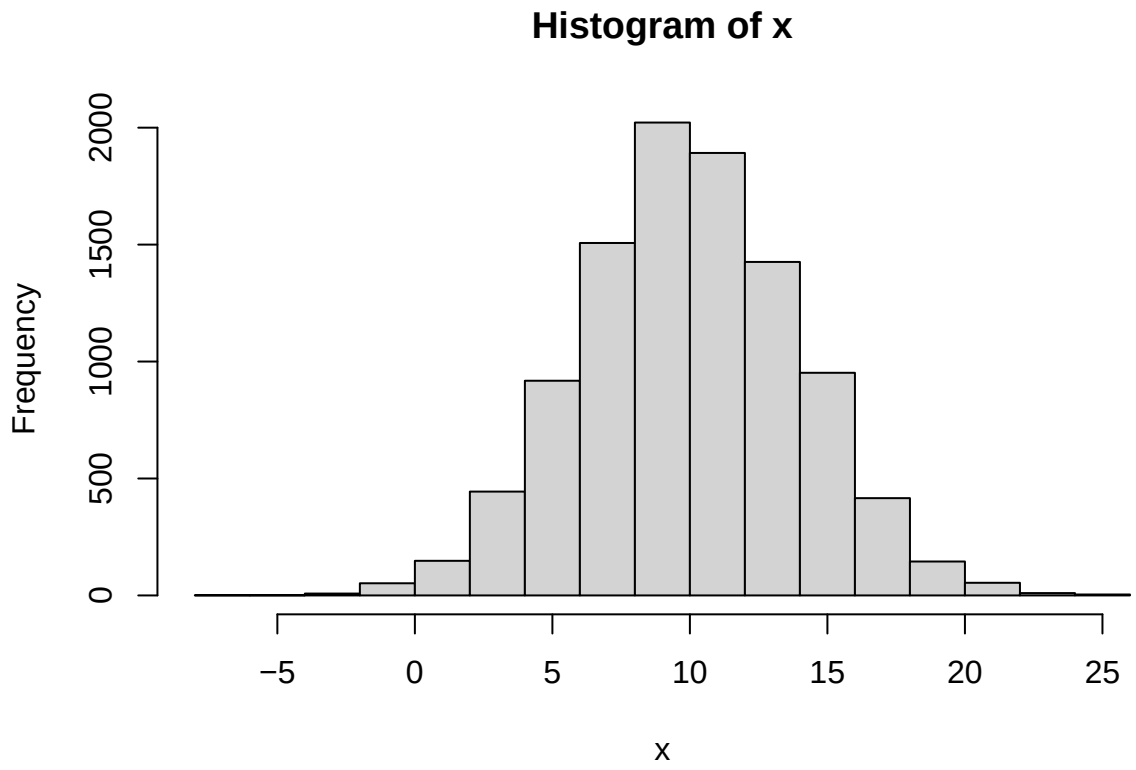
```
rep(rep((seq(from=0.2, to=0.6, by=0.2)), each=2), times=3)
```

```
##  [1] 0.2 0.2 0.4 0.4 0.6 0.6 0.2 0.2 0.4 0.4 0.6 0.6 0.2 0.2 0.4 0.4 0.6 0.6
```

```
### For many distributions there are four functions (example normal):
# rnorm            # generate pseudorandom numbers
# dnorm            # density values
```

```r
# pnorm            # CDF
# qnorm            # quantiles

# Ex 4: a. Generate a sample of 10000 normal values with mean 10 and sd 4
#       b. Make a histogram of the values
#       c. Check that the empirical mean and SD match those used to generate the sample
#       d. For X~N(10, 16), find Pr(X > 18)
#       e. For X~N(10, 16), find the value so that 97.5% of the distribution is less than that value
#       f. Make a smooth line graph of the N(10, 16) density.  Don't superimpose on the histogram.
#          (Hint: Use seq)
x <- rnorm(10000, mean=10, sd=4)
hist(x)
```




```r
mean(x)
```

```
## [1] 9.959305
```

```r
sd(x)
```

```
## [1] 3.959776
```

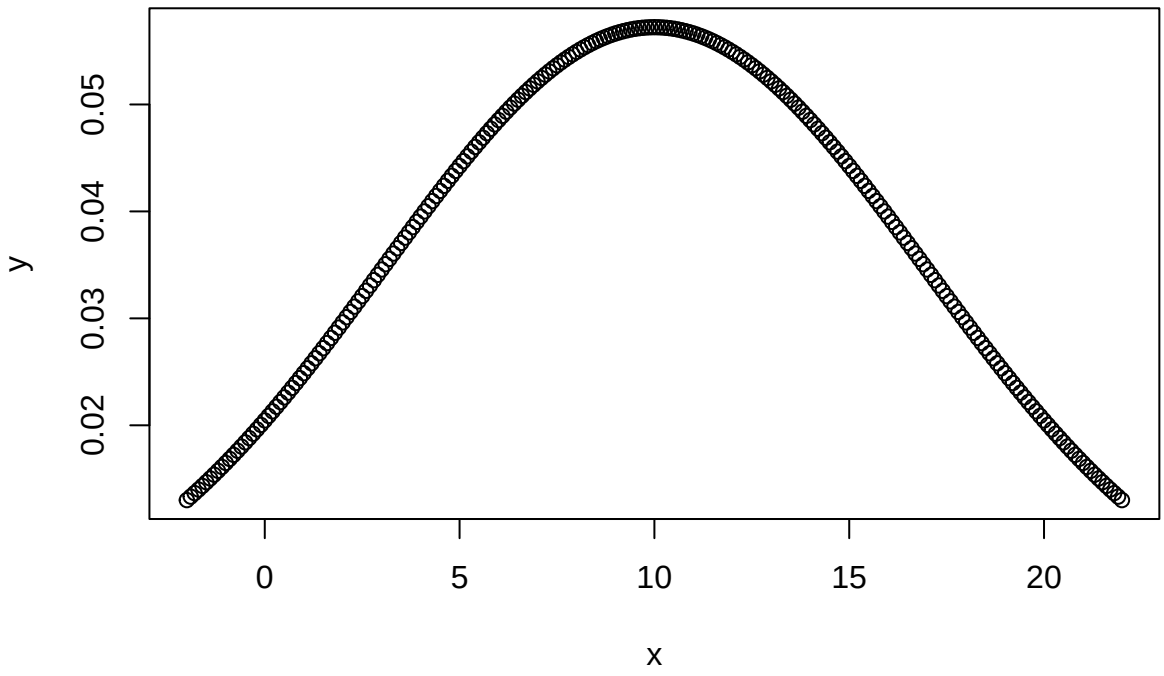
```r
pnorm(18, mean=10, sd=4, lower.tail=FALSE)
```

```
## [1] 0.02275013
```

```r
qnorm(.975, mean=10, sd=4)
```

```
## [1] 17.83986
```

```r
x= seq(-2, 22, by=0.1)
y= dnorm(x, mean(x), sd(x))
plot(x,y)
```




```r
# Ex 5: Generate a sample of 10000 values from a Gamma distribution with mean 10 and sd 4
#       Verify empirically that your sample mean and sd are close to 10 and 4
#       This is very useful, when working with a new distn in R (or SAS or ...) it's good to do this
#  10=alpha*theta, 4=sqrt(alpha)*theta, theta=10/alpha=4/sqrt(alpha),  alpha=100/16, theta= 10/alpha)
shape <- 100/16
scale <- 10/shape
z <- rgamma(10000, shape=shape, scale=scale)
mean(z)
```

```
## [1] 10.0402
```

```r
sd(z)
```

```
## [1] 4.061288
```

```r
# Ex 6: Explain (1 sentence each) what each of these statements does

x[c(3:7)-2]
```

```
## [1] -2.0 -1.9 -1.8 -1.7 -1.6
```

```r
x[c(3:7)]-2
```

```
## [1] -3.8 -3.7 -3.6 -3.5 -3.4
```

```r
#xy.mat[14, ]
xy.mat[c(2:4)]
```

```
## [1] 3 4 6
```

```r
cbind(x, lab)
```

```
## Warning in cbind(x, lab): number of rows of result is not a multiple of vector length (arg 2)
```

```
##         x                     lab
##    [1,] "-2"                  "a"
##    [2,] "-1.9"                "b"
##    [3,] "-1.8"                "c"
##    [4,] "-1.7"                "d"
##    [5,] "-1.6"                "e"
##    [6,] "-1.5"                "f"
##    [7,] "-1.4"                "g"
##    [8,] "-1.3"                "h"
##    [9,] "-1.2"                "a"
##   [10,] "-1.1"                "b"
##   [11,] "-1"                  "c"
##   [12,] "-0.9"                "d"
##   [13,] "-0.8"                "e"
##   [14,] "-0.7"                "f"
##   [15,] "-0.6"                "g"
##   [16,] "-0.5"                "h"
##   [17,] "-0.4"                "a"
##   [18,] "-0.3"                "b"
##   [19,] "-0.2"                "c"
##   [20,] "-0.0999999999999999" "d"
##   [21,] "0"                   "e"
##   [22,] "0.1"                 "f"
##   [23,] "0.2"                 "g"
##   [24,] "0.3"                 "h"
##   [25,] "0.4"                 "a"
##   [26,] "0.5"                 "b"
##   [27,] "0.6"                 "c"
##   [28,] "0.7"                 "d"
##   [29,] "0.8"                 "e"
##   [30,] "0.9"                 "f"
##   [31,] "1"                   "g"
```

```
##  [32,] "1.1"               "h"
##  [33,] "1.2"               "a"
##  [34,] "1.3"               "b"
##  [35,] "1.4"               "c"
##  [36,] "1.5"               "d"
##  [37,] "1.6"               "e"
##  [38,] "1.7"               "f"
##  [39,] "1.8"               "g"
##  [40,] "1.9"               "h"
##  [41,] "2"                 "a"
##  [42,] "2.1"               "b"
##  [43,] "2.2"               "c"
##  [44,] "2.3"               "d"
##  [45,] "2.4"               "e"
##  [46,] "2.5"               "f"
##  [47,] "2.6"               "g"
##  [48,] "2.7"               "h"
##  [49,] "2.8"               "a"
##  [50,] "2.9"               "b"
##  [51,] "3"                 "c"
##  [52,] "3.1"               "d"
##  [53,] "3.2"               "e"
##  [54,] "3.3"               "f"
##  [55,] "3.4"               "g"
##  [56,] "3.5"               "h"
##  [57,] "3.6"               "a"
##  [58,] "3.7"               "b"
##  [59,] "3.8"               "c"
##  [60,] "3.9"               "d"
##  [61,] "4"                 "e"
##  [62,] "4.1"               "f"
##  [63,] "4.2"               "g"
##  [64,] "4.3"               "h"
##  [65,] "4.4"               "a"
##  [66,] "4.5"               "b"
##  [67,] "4.6"               "c"
##  [68,] "4.7"               "d"
##  [69,] "4.8"               "e"
##  [70,] "4.9"               "f"
##  [71,] "5"                 "g"
##  [72,] "5.1"               "h"
##  [73,] "5.2"               "a"
##  [74,] "5.3"               "b"
##  [75,] "5.4"               "c"
##  [76,] "5.5"               "d"
##  [77,] "5.6"               "e"
##  [78,] "5.7"               "f"
##  [79,] "5.8"               "g"
##  [80,] "5.9"               "h"
##  [81,] "6"                 "a"
##  [82,] "6.1"               "b"
##  [83,] "6.2"               "c"
##  [84,] "6.3"               "d"
##  [85,] "6.4"               "e"
```

```
##  [86,] "6.5"                "f"
##  [87,] "6.6"                "g"
##  [88,] "6.7"                "h"
##  [89,] "6.8"                "a"
##  [90,] "6.9"                "b"
##  [91,] "7"                  "c"
##  [92,] "7.1"                "d"
##  [93,] "7.2"                "e"
##  [94,] "7.3"                "f"
##  [95,] "7.4"                "g"
##  [96,] "7.5"                "h"
##  [97,] "7.6"                "a"
##  [98,] "7.7"                "b"
##  [99,] "7.8"                "c"
## [100,] "7.9"                "d"
## [101,] "8"                  "e"
## [102,] "8.1"                "f"
## [103,] "8.2"                "g"
## [104,] "8.3"                "h"
## [105,] "8.4"                "a"
## [106,] "8.5"                "b"
## [107,] "8.6"                "c"
## [108,] "8.7"                "d"
## [109,] "8.8"                "e"
## [110,] "8.9"                "f"
## [111,] "9"                  "g"
## [112,] "9.1"                "h"
## [113,] "9.2"                "a"
## [114,] "9.3"                "b"
## [115,] "9.4"                "c"
## [116,] "9.5"                "d"
## [117,] "9.6"                "e"
## [118,] "9.7"                "f"
## [119,] "9.8"                "g"
## [120,] "9.9"                "h"
## [121,] "10"                 "a"
## [122,] "10.1"               "b"
## [123,] "10.2"               "c"
## [124,] "10.3"               "d"
## [125,] "10.4"               "e"
## [126,] "10.5"               "f"
## [127,] "10.6"               "g"
## [128,] "10.7"               "h"
## [129,] "10.8"               "a"
## [130,] "10.9"               "b"
## [131,] "11"                 "c"
## [132,] "11.1"               "d"
## [133,] "11.2"               "e"
## [134,] "11.3"               "f"
## [135,] "11.4"               "g"
## [136,] "11.5"               "h"
## [137,] "11.6"               "a"
## [138,] "11.7"               "b"
## [139,] "11.8"               "c"
```

```
## [140,] "11.9"              "d"
## [141,] "12"                "e"
## [142,] "12.1"              "f"
## [143,] "12.2"              "g"
## [144,] "12.3"              "h"
## [145,] "12.4"              "a"
## [146,] "12.5"              "b"
## [147,] "12.6"              "c"
## [148,] "12.7"              "d"
## [149,] "12.8"              "e"
## [150,] "12.9"              "f"
## [151,] "13"                "g"
## [152,] "13.1"              "h"
## [153,] "13.2"              "a"
## [154,] "13.3"              "b"
## [155,] "13.4"              "c"
## [156,] "13.5"              "d"
## [157,] "13.6"              "e"
## [158,] "13.7"              "f"
## [159,] "13.8"              "g"
## [160,] "13.9"              "h"
## [161,] "14"                "a"
## [162,] "14.1"              "b"
## [163,] "14.2"              "c"
## [164,] "14.3"              "d"
## [165,] "14.4"              "e"
## [166,] "14.5"              "f"
## [167,] "14.6"              "g"
## [168,] "14.7"              "h"
## [169,] "14.8"              "a"
## [170,] "14.9"              "b"
## [171,] "15"                "c"
## [172,] "15.1"              "d"
## [173,] "15.2"              "e"
## [174,] "15.3"              "f"
## [175,] "15.4"              "g"
## [176,] "15.5"              "h"
## [177,] "15.6"              "a"
## [178,] "15.7"              "b"
## [179,] "15.8"              "c"
## [180,] "15.9"              "d"
## [181,] "16"                "e"
## [182,] "16.1"              "f"
## [183,] "16.2"              "g"
## [184,] "16.3"              "h"
## [185,] "16.4"              "a"
## [186,] "16.5"              "b"
## [187,] "16.6"              "c"
## [188,] "16.7"              "d"
## [189,] "16.8"              "e"
## [190,] "16.9"              "f"
## [191,] "17"                "g"
## [192,] "17.1"              "h"
## [193,] "17.2"              "a"
```

```
## [194,] "17.3"               "b"
## [195,] "17.4"               "c"
## [196,] "17.5"               "d"
## [197,] "17.6"               "e"
## [198,] "17.7"               "f"
## [199,] "17.8"               "g"
## [200,] "17.9"               "h"
## [201,] "18"                 "a"
## [202,] "18.1"               "b"
## [203,] "18.2"               "c"
## [204,] "18.3"               "d"
## [205,] "18.4"               "e"
## [206,] "18.5"               "f"
## [207,] "18.6"               "g"
## [208,] "18.7"               "h"
## [209,] "18.8"               "a"
## [210,] "18.9"               "b"
## [211,] "19"                 "c"
## [212,] "19.1"               "d"
## [213,] "19.2"               "e"
## [214,] "19.3"               "f"
## [215,] "19.4"               "g"
## [216,] "19.5"               "h"
## [217,] "19.6"               "a"
## [218,] "19.7"               "b"
## [219,] "19.8"               "c"
## [220,] "19.9"               "d"
## [221,] "20"                 "e"
## [222,] "20.1"               "f"
## [223,] "20.2"               "g"
## [224,] "20.3"               "h"
## [225,] "20.4"               "a"
## [226,] "20.5"               "b"
## [227,] "20.6"               "c"
## [228,] "20.7"               "d"
## [229,] "20.8"               "e"
## [230,] "20.9"               "f"
## [231,] "21"                 "g"
## [232,] "21.1"               "h"
## [233,] "21.2"               "a"
## [234,] "21.3"               "b"
## [235,] "21.4"               "c"
## [236,] "21.5"               "d"
## [237,] "21.6"               "e"
## [238,] "21.7"               "f"
## [239,] "21.8"               "g"
## [240,] "21.9"               "h"
## [241,] "22"                 "a"
```

```
# Ex 7: Using the object xy.mat and not making any new assignments (don't use <-),
#       graph y2 versus x, omitting the 4th row, and with axis labels "y2" and "x"
```